

Production Web Application Outage

Interview Preparation Guide

Author: Naftali Wodinsky

Ticket: 002

Focus: Investigation, Analysis, Containment and Recovery

Document Date: July 30, 2026

What was your first hypothesis?

My first thought was that something had failed somewhere along the user-facing request path.

At that stage, I did not assume that Nginx, DNS, networking, Docker, Kubernetes, or the Flask application was responsible. The ticket only told me that users could no longer reach the application, so I kept the initial scope broad.

Why did you start there?

A browser error can originate from several different parts of the environment, so I began with the full request path:

```
Client -> Network -> TCP port 80 -> Nginx -> Flask backend
```

Looking at the entire path helped me avoid making changes before I knew which component had actually failed.

My first checks were:

1. Confirm that the Ubuntu server was reachable.
2. Test TCP port 80.
3. Check the Nginx service.
4. Review the listening ports.
5. Test the Flask backend directly.

What evidence caused you to change direction?

The Ubuntu server was still reachable, and SSH continued working, but TCP port 80 was unavailable.

I then confirmed that Nginx was not running while the Flask backend on `127.0.0.1:5050` was still returning HTTP 200.

That narrowed the investigation considerably. Instead of treating the event as a general server, network, or application outage, I began looking specifically at why Nginx had stopped.

The Nginx journal showed a clean shutdown rather than a crash. Authentication and sudo records then showed that `webadmin` had executed:

```
/usr/bin/systemctl stop nginx
```

That was the point where the investigation shifted toward privileged account activity.

What evidence ruled out the other possibilities?

I ruled out a complete server failure because:

- The host was reachable.
- SSH was available.
- Other services were still running.
- The operating system had not crashed.

I ruled out a broader network failure because:

- The host responded to network testing.
- TCP port 22 was reachable.
- The problem was limited to the web service.

I ruled out DNS because:

- I tested the application directly by IP address.
- Local loopback testing did not rely on DNS.

I ruled out a Flask backend failure because:

- `company-web.service` was active.
- Port 5050 returned HTTP 200.
- The backend did not need to be restarted.

I ruled out an Nginx configuration problem because:

- `nginx -t` passed.
- The same configuration worked after the restart.
- I did not need to edit the configuration.

I ruled out an Nginx crash because:

- Systemd recorded a normal stop request.
- There was no crash, segmentation fault, or resource-failure message.

I ruled out Docker and Kubernetes because:

- The application ran directly on the Ubuntu host.
- Nginx and Flask were managed by systemd.
- Neither Docker nor Kubernetes was part of the application request path.

What monitoring would have detected the outage sooner?

Several controls could have identified the outage much earlier:

- External HTTP checks against the user-facing application
- TCP port 80 monitoring
- Alerts when Nginx becomes inactive
- Alerts when the application stops returning HTTP 200

- SIEM alerts for privileged service-control commands
- Alerts for unusual privileged SSH logins
- Alerts for failed or repeated sudo authentication
- Centralized monitoring of systemd service changes

The most useful control would have been an external HTTP check because it would have tested the same path the users depended on.

What would you do differently in production?

In a production environment, I would follow a formal incident-response process.

I would:

1. Open or update the incident record and assign a severity.
2. Preserve volatile evidence before making unnecessary changes.
3. Notify the incident commander, application owner, security team, and business stakeholders.
4. Confirm the full scope of affected users and systems.
5. Review identity-provider, VPN, bastion, endpoint, firewall, and SIEM records.
6. Suspend the involved account through the identity platform.
7. Revoke active sessions, passwords, SSH keys, tokens, and certificates.
8. Isolate and investigate the source endpoint if credential misuse was suspected.
9. Restore service through an approved emergency-change process.
10. Test recovery from more than one external location.
11. Increase monitoring for repeated activity.
12. Preserve evidence according to chain-of-custody requirements.
13. Review privileged access across the environment.
14. Complete a post-incident review and assign corrective actions.

I would not rebuild the server unless I found evidence of malware, persistence, unauthorized file changes, or a loss of system integrity.

How did you confirm the root cause?

I compared several independent sources:

- SSH authentication records
- The remote source IP address
- Failed and successful sudo activity
- The exact privileged command
- The Nginx shutdown time
- Service status

- Listening-port output
- Direct backend testing
- Recovery results after restarting Nginx

The sudo-command timestamp matched the Nginx shutdown. Restarting Nginx restored the application without any changes to the backend, network, DNS, Docker, Kubernetes, or configuration.

Together, those findings confirmed the direct technical cause.

Was the application compromised?

I found no evidence that the Flask application itself had been compromised.

There was no evidence of:

- Application-file changes
- Malicious backend code
- Data theft
- Data alteration
- Persistence
- Backend process replacement

The event involved privileged activity that caused an availability outage. That is security-relevant, but it does not prove that the Flask application was compromised.

Was the activity malicious?

The technical evidence shows that the `webadmin` account ran the command that stopped Nginx.

It does not show whether the action was:

- Accidental
- Deliberate
- Performed by the legitimate account owner
- Performed by someone using shared or stolen credentials

Without an approved change record, I would describe the action as unauthorized or inappropriate. I would not claim malicious intent without additional evidence.

Why did you lock the account before restoring service?

The account still had an active session and had already been used to interrupt the service.

If I had restarted Nginx without first containing the account, the same session could have stopped the service again or carried out other privileged actions.

Locking the account and ending the session reduced the immediate risk before I restored access.

How did you confirm that recovery was complete?

I checked the recovery at several levels:

- Nginx was active.
- The Flask backend was active.
- TCP port 80 was listening.
- The application returned HTTP 200 through Nginx.
- The Windows workstation could connect to port 80.
- The application opened successfully in a browser.
- Both services returned after reboot.
- The `webadmin` account remained locked after reboot.

These checks confirmed that the services were running, users could reach the application, and the recovery survived a restart.

What was the most important lesson?

The most important lesson was to test each layer independently and allow the evidence to guide the investigation.

The backend was healthy, the server was online, and the network was working. Users still experienced a complete outage because the reverse proxy had been stopped.

A structured investigation kept me from changing healthy components and helped me reach the correct conclusion faster.